

UNIT 4: Firewall and Defense

1. Introduction to Firewalls

Definition:

A **firewall** is a critical network security device or software application that acts as a gatekeeper between networks, or between different segments within a network. Its fundamental role is to **monitor and filter incoming and outgoing network traffic** based on a predefined set of **security policies**. It determines what traffic is allowed to pass through and what traffic is blocked, thereby enforcing the organization's security posture and protecting internal resources from external threats, and sometimes internal threats from spreading.

Firewalls operate at various layers of the OSI model, with more advanced firewalls incorporating intelligence from higher layers.

Types of Firewalls:

Firewalls have evolved significantly over time, becoming more sophisticated in their capabilities. Here's a detailed look at the common types:

1. Packet Filtering Firewall (Stateless Firewall):

- **Description:** This is the oldest and most basic type of firewall. It operates primarily at the **network layer (Layer 3) and transport layer (Layer 4)** of the OSI model. It inspects individual network packets in isolation, examining only the **headers** of each packet. It makes decisions based on a predefined set of rules that typically include:
 - Source IP address
 - Destination IP address
 - Source Port number
 - Destination Port number
 - Protocol (TCP, UDP, ICMP, etc.)
 - Direction (inbound or outbound)
- **How it works:** Each packet is evaluated against the rule set. If a packet matches a rule, the specified action (allow or deny) is taken. If no rule matches, a default action (usually deny) is applied. Crucially, a packet filtering firewall *does not* keep track of the state of a connection. For example, if an internal host initiates an outbound connection, this firewall will allow the initial outbound packets. However, for the return traffic, it would need a separate rule explicitly allowing inbound traffic from that

specific external IP/port, even though it's part of an already established connection.

- **Advantages:** Simple, fast, relatively inexpensive, low resource consumption.
- **Disadvantages:**
 - **Stateless:** Cannot determine if a packet is part of an existing, legitimate connection. This means it's vulnerable to spoofing attacks and makes it difficult to secure complex applications that use dynamic ports.
 - **Limited Context:** Only looks at header information, not the actual content of the packet. Cannot detect application-layer attacks.
 - **Rule Complexity:** Large rule sets can be difficult to manage and prone to errors.
- **Use Cases:** Often found as a component in home routers or as a basic security layer.

2. Stateful Inspection Firewall (Circuit-Level Gateway):

- **Description:** This type of firewall is a significant improvement over packet filters. It operates at the **network and transport layers** but, unlike packet filters, it **tracks the state of active network connections**. It maintains a "state table" that records information about established connections (e.g., source IP, destination IP, source port, destination port, sequence numbers).
- **How it works:** When a new connection request arrives, the firewall checks its state table. If it's a legitimate new connection (e.g., an internal host initiating an outbound connection), the firewall creates an entry in its state table. Subsequent packets belonging to that *same connection* (both inbound and outbound) are automatically allowed without needing explicit rules for return traffic. This makes it much more efficient and secure.
- **Advantages:**
 - **Enhanced Security:** More secure than packet filters because it only allows return traffic for connections that were legitimately initiated. Prevents certain types of spoofing and fragmentation attacks.
 - **Simpler Rule Sets:** No need to create explicit "allow return traffic" rules.

- **Better Performance:** Once a connection is established, subsequent packets are quickly matched against the state table.
- **Disadvantages:** Still primarily operates at lower layers, so it generally doesn't inspect application-layer content. Can be fooled by sophisticated application-layer attacks.
- **Use Cases:** The most common type of firewall found in corporate networks today, forming the backbone of perimeter defense.

3. Application Layer Firewall (Proxy Firewall / Gateway Firewall):

- **Description:** This firewall operates at the **application layer (Layer 7)** of the OSI model. It functions as an **intermediary (proxy)** between the client and the server. Instead of just forwarding packets, it actually intercepts and terminates the connection from the client, inspects the application-layer traffic, and then establishes a *separate* connection to the destination server.
- **How it works:** Because it acts as a proxy, it can thoroughly inspect the content of the traffic for specific commands, malicious payloads, or violations of application-specific protocols. For example, an application layer firewall can understand HTTP requests and responses, and even validate if the syntax of an HTTP request is correct or if it contains SQL injection attempts.
- **Advantages:**
 - **Highest Level of Security:** Deep packet inspection allows detection of application-layer attacks that other firewalls miss.
 - **Protocol-Awareness:** Can enforce strict protocol compliance.
 - **Anonymity:** Can mask the internal IP addresses of clients.
 - **Auditing:** Can log application-specific events in detail.
- **Disadvantages:**
 - **Performance Overhead:** Acting as a proxy adds latency and requires more processing power.
 - **Complexity:** More difficult to configure and manage.
 - **Application Specific:** Needs to understand specific protocols, so it may not support all applications.

- **Use Cases:** Often used for specific applications like web servers, email servers, or for highly secure environments where deep inspection is critical. Sometimes deployed as a reverse proxy.

4. Next-Generation Firewall (NGFW):

- **Description:** NGFWs are not a single technology but a category of firewalls that combine the capabilities of traditional firewalls (like stateful inspection) with advanced security features that were previously found in separate security devices. They aim to provide a more comprehensive, integrated security solution.
- **Key Features and How they Work:**
 - **Deep Packet Inspection (DPI):** Similar to application layer firewalls, NGFWs perform DPI to analyze the actual content of packets, not just headers, to identify malware, intrusion attempts, and policy violations across various applications and protocols.
 - **Integrated IDS/IPS (Intrusion Detection System/Intrusion Prevention System):** NGFWs incorporate the functionality of IDPS, allowing them to detect known attack signatures and block malicious traffic in real-time.
 - **Application Awareness and Control:** Can identify and control applications regardless of the port they use. For instance, an NGFW can differentiate between Facebook chat and Facebook games, allowing one while blocking the other.
 - **User Identity Awareness:** Integrates with directory services (like Active Directory) to apply security policies based on individual users or groups, rather than just IP addresses. This allows for more granular control ("John Doe can access this resource, but Jane Smith cannot").
 - **Threat Intelligence Integration:** Can pull in real-time threat intelligence feeds about malicious IPs, URLs, and malware signatures to enhance detection capabilities.
 - **URL Filtering/Web Filtering:** Can block access to specific websites or categories of websites (e.g., gambling, adult content).
 - **Malware Prevention:** Often includes anti-malware and sandboxing capabilities to detect and prevent known and zero-day malware.

- **Advantages:** Offers multi-layered protection, reduces the need for multiple discrete security appliances, provides better visibility and control, more effective against advanced threats.
- **Disadvantages:** High cost, complex to deploy and manage, can introduce latency if not properly sized.
- **Use Cases:** Increasingly becoming the standard for enterprise network security, offering a comprehensive defense against modern, multi-vector attacks.

5. Proxy Firewall:

- **Description:** As mentioned under Application Layer Firewalls, a proxy firewall is essentially an **application layer gateway**. It acts as an intermediary, breaking the direct connection between internal clients and external servers.
- **How it works:**
 1. A client makes a request to the proxy firewall.
 2. The proxy firewall receives the request, inspects it at the application layer, and determines if it's permitted by policy.
 3. If allowed, the proxy firewall makes a *new* request on behalf of the client to the destination server.
 4. The server responds to the proxy firewall.
 5. The proxy firewall inspects the server's response.
 6. If allowed, the proxy firewall forwards the response back to the client.
- **Advantages:** Strong security (deep inspection, content filtering), masks internal network topology, caching capabilities (for web proxies) can improve performance.
- **Disadvantages:** Performance overhead, complexity, protocol specificity.
- **Use Cases:** Web proxies, reverse proxies (protecting web servers), FTP proxies. Often a core component of NGFWs.

Firewall Rules:

Firewall rules are the instructions that dictate how a firewall processes network traffic. They are typically evaluated in a sequential order, from top to bottom. Once a packet matches a rule, the specified action is taken, and no further rules are evaluated for that packet.

Each rule generally includes the following key components:

- **Source IP (Internet Protocol) Address:**
 - **Description:** Specifies the origin of the network traffic. This can be a single IP address (e.g., 192.168.1.10), an IP address range, a subnet (e.g., 192.168.1.0/24), or a wildcard representing "any" source.
 - **Purpose:** To control traffic *from* specific devices, networks, or the internet.
 - **Example:** Only allow traffic originating from your internal office network.
- **Destination IP Address:**
 - **Description:** Specifies the intended recipient of the network traffic. Similar to the source IP, this can be a single IP, a range, a subnet, or "any" destination.
 - **Purpose:** To control traffic *to* specific servers, devices, or external destinations.
 - **Example:** Block all traffic destined for known malicious IP addresses.
- **Port Number:**
 - **Description:** Specifies the logical port number associated with a specific service or application (e.g., 80 for HTTP, 443 for HTTPS, 22 for SSH). This can be a single port, a range of ports, or "any" port.
 - **Purpose:** To allow or deny access to specific services running on the destination device.
 - **Example:** Allow inbound traffic only to port 443 on your web server, blocking all other ports.
- **Protocol:**
 - **Description:** Specifies the transport layer protocol used (most commonly TCP, UDP, or ICMP).
 - **Purpose:** To differentiate between different types of traffic that might use the same port but require different handling (e.g., DNS uses both TCP and UDP on port 53).
- **Action:**
 - **Description:** The action the firewall takes when a packet matches the rule. The two primary actions are:

- **Allow (or Permit):** The traffic is permitted to pass through the firewall.
- **Deny (or Drop/Block):** The traffic is discarded by the firewall.
- **Optional Actions:** Some firewalls might also have actions like:
 - **Reject:** Deny the traffic and send an ICMP "destination unreachable" message back to the source, indicating the traffic was actively refused. This can be useful for troubleshooting but can also provide information to an attacker.
 - **Log:** Record the event in the firewall's logs, regardless of whether it's allowed or denied. This is crucial for auditing and security monitoring.
- **Implicit Deny:** A fundamental security principle for firewalls is the "implicit deny" rule. This means that if a packet does not explicitly match any "allow" rule, it is automatically denied by default. This ensures that only explicitly permitted traffic can pass.

Example Firewall Rule (Conceptual):

ALLOW TCP FROM 192.168.1.0/24 TO 172.16.0.10 ON PORT 443 LOG

- **Meaning:** Allow TCP traffic originating from any device in the 192.168.1.0/24 subnet, destined for the server at 172.16.0.10, specifically on port 443 (HTTPS). Also, log this activity.

Firewall rules are the heart of a firewall's operation and require careful planning and regular review to maintain effective network security.

2. NIC Promiscuous Mode

Definition:

Promiscuous mode is a special operational setting for a Network Interface Card (NIC), often referred to as an Ethernet adapter. In its default (non-promiscuous or "unicast") mode, a NIC is designed to process only those network packets whose destination MAC address matches its own MAC address, broadcast packets (to all devices), and multicast packets (to a group of devices it belongs to).

When a NIC is configured in **promiscuous mode**, it overrides this default behavior. Instead of selectively processing packets, it intercepts and delivers *all* network packets it sees on the connected network segment (collision domain or broadcast domain) to the

operating system's software stack, **regardless of their destination MAC address**. This means it can "listen in" on conversations not explicitly addressed to it.

Usage in Security:

Promiscuous mode is an invaluable tool for network administrators, security analysts, and forensic investigators. While it can be exploited maliciously, its legitimate uses are critical for network health and security:

- **Packet Sniffing:** This is the most direct application. By putting a NIC in promiscuous mode, a tool like Wireshark (or tcpdump) can capture all traffic on a local network segment. This allows for:
 - **Traffic Analysis:** Understanding the types of protocols and data flowing across the network.
 - **Troubleshooting:** Identifying network bottlenecks, misconfigurations, or application communication issues by observing the actual packets being exchanged.
 - **Security Auditing:** Checking for unencrypted protocols transmitting sensitive data (e.g., plaintext passwords), detecting unauthorized services, or identifying unusual network behavior.
- **Network Monitoring:** Promiscuous mode is essential for network monitoring systems that need to collect data about overall network usage, performance, and health.
 - **Bandwidth Monitoring:** Tools can analyze packet sizes and frequencies to identify bandwidth hogs.
 - **Application Performance Monitoring:** Observing latency and retransmissions for specific applications.
 - **Anomaly Detection:** Establishing a baseline of "normal" traffic and alerting when significant deviations occur.
- **Intrusion Detection (NIDS):** Network-based Intrusion Detection Systems (NIDS) rely heavily on promiscuous mode. An NIDS sensor is typically deployed on a network segment (often using a network tap or SPAN/mirror port on a switch) with its NIC in promiscuous mode. It then captures all traffic and analyzes it for known attack signatures or anomalous patterns, generating alerts when suspicious activity is detected.
- **Forensics (Digital Forensics and Incident Response - DFIR):**
 - **Post-incident Analysis:** After a security incident (e.g., a breach, malware infection), network traffic captures taken in promiscuous mode can be

invaluable. They provide a "flight recorder" of network communications that occurred during the incident, allowing investigators to:

- Determine the initial access vector.
 - Track lateral movement of attackers.
 - Identify command and control (C2) communications.
 - Discover data exfiltration attempts.
 - Understand the attacker's tools and techniques.
- **Evidence Collection:** Network captures can serve as crucial evidence in legal or compliance investigations.

Enabling Promiscuous Mode (Linux):

Enabling promiscuous mode requires root privileges on Linux systems. Here are the common commands:

- **Using ifconfig (Legacy):**

Bash

`ifconfig eth0 promisc`

- `ifconfig`: A command-line utility for network interface configuration. While still present on many systems, it's considered deprecated in favor of `ip`.
- `eth0`: Replace `eth0` with the actual name of your network interface (e.g., `enp0s3`, `wlan0`).
- `promisc`: The keyword that enables promiscuous mode.
- To disable: `ifconfig eth0 -promisc`

- **Using ip link (Modern/Preferred):**

`ip link set eth0 promisc on`

- `ip link`: Part of the `iproute2` suite, this is the modern and preferred tool for network interface management in Linux.
- `set eth0`: Specifies the network interface to configure.
- `promisc on`: Enables promiscuous mode.
- To disable: `ip link set eth0 promisc off`

Verification: You can check the status of promiscuous mode for an interface using:

`ip link show eth0` (look for `PROMISC` in the flags)

or

`cat /sys/class/net/eth0/flags` (bitwise AND with 0x100 for promiscuous flag).

In Wireshark:

Wireshark, being a packet analyzer, inherently needs to use promiscuous mode to capture all traffic on a segment. It provides a straightforward option within its graphical user interface:

1. **Go to Capture > Options:** This menu opens the "Wireshark Capture Interfaces" dialog.
2. **Check the box: Enable promiscuous mode on all interfaces:** This global setting ensures that any interface you select for capture will have promiscuous mode enabled.
 - Alternatively, you can select specific interfaces and enable promiscuous mode for them individually if you only want to capture traffic on certain segments.

Important Note on Switched Networks:

On modern switched networks, promiscuous mode on a regular workstation NIC will primarily capture:

- Traffic to and from the workstation itself.
- Broadcast traffic.
- Multicast traffic.
- Traffic for devices on the same physical port if a hub is used (rare in corporate environments).
- ****Crucially, it will *not* capture traffic between other devices connected to the same switch unless the switch port is configured for **SPAN (Switched Port Analyzer) or port mirroring** mode, which duplicates all traffic from other ports to the port where the promiscuous mode NIC is connected. This is a common setup for NIDS sensors.**

3. Introduction to Wireshark

What is Wireshark?

Wireshark is the world's most widely used and powerful **open-source packet analyzer** (also known as a network protocol analyzer or network sniffer). It was originally named **Ethereal** but was rebranded **Wireshark** in 2006.

Its core function is to **capture and display real-time network traffic in extreme detail**. It presents raw packet data in a human-readable format, making it indispensable for anyone needing to understand exactly what's happening on a network wire.

Essential for:

- **Network Troubleshooting:** Identifying the root cause of network connectivity issues, slow performance, application failures, and misconfigurations by examining the actual data flow.
- **Network Analysis:** Gaining deep insights into network behavior, protocol implementation, and traffic patterns. This includes capacity planning, security auditing, and understanding how different applications communicate.
- **Digital Forensics:** As mentioned previously, Wireshark is a critical tool for incident response, allowing analysts to reconstruct attack chains, identify malicious payloads, and collect evidence after a security breach.
- **Protocol Development/Debugging:** For developers working on network applications or new protocols, Wireshark helps verify proper protocol implementation.
- **Security Education:** It's an excellent tool for learning about network protocols and security concepts in a practical, hands-on manner.

Key Features:

Wireshark's extensive feature set makes it highly versatile:

- **Deep Protocol Analysis (Protocol Dissectors):** This is Wireshark's standout feature. It understands hundreds of network protocols (from Ethernet and IP to application-layer protocols like HTTP, DNS, SMB, DICOM, industrial protocols, etc.). It can dissect (decode and interpret) the various fields within each protocol header and payload, presenting them in a structured, hierarchical view. This saves users from manually decoding raw hexadecimal data.
- **Real-time Capture and Offline Analysis:** It can capture live traffic directly from network interfaces or open saved capture files (e.g., .pcap, .pcapng) for offline analysis. This flexibility is crucial for both immediate troubleshooting and post-incident investigations.
- **Packet Filtering (Capture and Display Filters):** Offers powerful filtering capabilities to isolate specific traffic of interest. This is essential given the sheer volume of data on modern networks.
 - **Capture Filters:** Filter packets *before* they are saved to the capture file, reducing file size and overhead.

- **Display Filters:** Filter packets *after* they have been captured, allowing for flexible, on-the-fly analysis without re-capturing.
- **Exporting Capture Data:** Allows users to save captured packets in various formats (e.g., .pcap, .pcapng, plain text, CSV, XML) for sharing, further analysis with other tools, or archiving. It can also export specific packet data or protocol fields.
- **Color-Coded Traffic:** Visually distinguishes different types of traffic (e.g., TCP errors, DNS queries, HTTP traffic) using customizable color rules. This makes it easier to spot anomalies or specific events quickly.
- **Packet Reassembly (TCP Streams):** Can reassemble fragmented IP packets and reconstruct entire TCP streams (the full conversation between two endpoints), allowing analysts to view the complete data exchange as if it were a single file or chat session. This is critical for analyzing application-layer data (e.g., web pages, file transfers).
- **Multiple Interface Support:** Can capture from Ethernet, Wi-Fi, Bluetooth, USB, Loopback, and other interfaces.
- **Command-Line Tools:** Comes with companion command-line tools like tshark (for terminal-based packet analysis) and editcap (for manipulating capture files).
- **Plugin Architecture:** Allows developers to extend Wireshark's capabilities by adding new protocol dissectors or analysis tools.

4. Configuring Wireshark

Effective use of Wireshark starts with proper configuration for your specific capture needs.

Basic Setup:

1. **Launch Wireshark:** Start the Wireshark application (usually from your desktop icon, start menu, or by typing wireshark in the terminal).
2. **Choose the correct network interface:**
 - On the Wireshark home screen, you'll see a list of available network interfaces (e.g., Ethernet, Wi-Fi, Loopback).
 - Each interface will often show a small graph indicating current activity. Select the interface through which the traffic you want to capture is flowing. For example, if you're connected via an Ethernet cable, choose

your eth0 or Ethernet adapter. If you're on Wi-Fi, select your wlan0 or Wi-Fi adapter.

- **Crucial Consideration:** If you're capturing on a switched network and want to see traffic *not* destined for your machine, ensure your network switch port is configured for SPAN/port mirroring to direct traffic to your chosen interface, or you are connected to a hub (less common).
3. **Enable promiscuous mode:** As discussed in section 2, this is vital for capturing all traffic on the segment your NIC sees.
- Go to **Capture > Options...** (or click the gear icon on the main screen).
 - In the "Wireshark Capture Interfaces" dialog, ensure the **"Enable promiscuous mode on all interfaces"** checkbox is selected. You can also select it for specific interfaces if preferred.
4. **Start capture:**
- Once your interface is selected and promiscuous mode is enabled, click the blue **"shark fin" icon** on the toolbar or press **Ctrl + E**.
 - Wireshark will immediately begin capturing packets in real-time and displaying them in the "Packet List Pane."

Settings:

Wireshark offers extensive configuration options to tailor your capture and analysis environment.

Capture Options (Before Starting Capture):

Accessible via **Capture > Options...** or the gear icon next to the interface list.

- **Select Interface(s):**
 - Allows you to choose one or more network interfaces from which to capture traffic. You can hold Ctrl to select multiple interfaces for simultaneous capture.
- **Set Capture Limit (Stop Conditions):**
 - Prevents capture files from growing too large and consuming excessive disk space or memory. You can set limits based on:
 - **After (X) packets:** Stop after a certain number of packets.
 - **After (X) megabytes/gigabytes:** Stop after the capture file reaches a certain size.

- **After (X) seconds/minutes/hours:** Stop after a specific duration.
- **When ring buffer fills:** For ring buffer captures, stop when the buffer is full (or overwrite oldest files).
- **Set Capture Filter (if required):**
 - **Description:** This is a crucial setting to reduce the amount of data captured. A capture filter instructs the NIC and capture driver to discard packets *before* they are even written to the capture file. This is highly efficient for targeted analysis.
 - **Syntax:** Capture filters use **BPF (Berkeley Packet Filter)** syntax. (More details in section 7).
 - **How to Apply:** In the "Capture Options" dialog, there's a "Capture Filter" field for each selected interface. You type your BPF filter string directly into this field.
 - **Example:** To capture only web traffic, you might enter port 80 or port 443.
 - **Advantage:** Reduces processing overhead and file size.
 - **Disadvantage:** If you filter out something you later realize you needed, you have to re-capture. Use with caution, especially in forensics.
- **Output Options:**
 - **File:** Specify a file name and location to save the captured packets.
 - **Use multiple files (ring buffer):** Configure Wireshark to save captures in a series of files, overwriting the oldest when a certain limit (size or time) is reached. This is useful for continuous monitoring, ensuring you always have the most recent data without filling up your disk.

Preferences (Customizing Wireshark's Behavior):

Accessible via **Edit > Preferences...**

- **Customize Font, Colors, Protocol Decoding:**
 - **Appearance:** Adjust fonts, sizes, and colors for better readability.
 - **Coloring Rules:** Define custom rules to highlight specific types of packets in the Packet List Pane (e.g., bold red for connection resets, green for successful HTTP requests). This is highly powerful for visual analysis.
 - **Protocols:** This section allows you to fine-tune how Wireshark dissects various protocols. For example, you can tell Wireshark to treat traffic on a non-standard port as a specific protocol (e.g., interpret traffic on port

12345 as HTTP if you know your application uses it). You can also enable/disable specific dissectors.

- **Layout:** Adjust the arrangement of the three main panes (Packet List, Packet Details, Packet Bytes).

- **Other Important Preferences:**

- **Name Resolution:** Configure whether Wireshark attempts to resolve IP addresses to hostnames (DNS lookups) or MAC addresses to vendor names. While useful for readability, live DNS lookups can slow down analysis or generate additional network traffic.
- **IO Graphs:** Settings for how data is plotted in the IO Graph feature (Statistics > IO Graphs).

Careful configuration ensures that Wireshark captures precisely the data you need and presents it in a way that facilitates efficient analysis.

5. The Capture Window

Once you start a capture in Wireshark, the main window transforms into the "Capture Window," which is divided into three primary panes (by default):

Components:

1. Packet List Pane (Top Pane):

- **Description:** This pane displays a summary of each captured packet in chronological order. Each row represents a single packet.
- **Columns (Configurable):**
 - **No.:** The sequential number of the packet in the capture file.
 - **Time:** The timestamp when the packet was captured (relative to the start of the capture or absolute time, configurable).
 - **Source:** The IP address or resolved hostname of the sender of the packet.
 - **Destination:** The IP address or resolved hostname of the recipient of the packet.
 - **Protocol:** The highest-level protocol that Wireshark could identify for that packet (e.g., HTTP, TCP, DNS, ARP).
 - **Length:** The size of the packet in bytes.

- **Info:** A brief, human-readable summary of the packet's contents, often showing key details of the highest-level protocol (e.g., "HTTP GET /index.html", "Standard query A www.example.com", "TCP segment of a reassembled PDU").
- **Interaction:** Clicking on a packet in this pane highlights it and populates the other two panes with its details.

2. Packet Details Pane (Middle Pane):

- **Description:** This pane provides a hierarchical, tree-like breakdown of the selected packet's contents. It dissects the packet layer by layer, according to the OSI model.
- **Structure:** Each expandable line represents a different protocol layer or a significant field within a layer.
 - **Frame:** Information about the captured frame itself (e.g., arrival time, capture length, interface).
 - **Ethernet (Layer 2):** Source and Destination MAC addresses, EtherType.
 - **IP (Layer 3):** Source and Destination IP addresses, protocol (e.g., TCP, UDP, ICMP), TTL (Time To Live), fragmentation info.
 - **TCP/UDP (Layer 4):** Source and Destination Port numbers, sequence and acknowledgment numbers (for TCP), flags (SYN, ACK, FIN, RST), window size, checksum.
 - **Application Layer (Layer 7):** Details specific to the application protocol (e.g., HTTP request headers, DNS query details, SMB commands).
- **Interaction:** Expanding a layer (by clicking the + or arrow) reveals its sub-fields. Selecting a field here often highlights the corresponding raw bytes in the Packet Bytes Pane. This is where you gain deep insights into protocol behavior.

3. Packet Bytes Pane (Bottom Pane):

- **Description:** This pane displays the raw data of the selected packet in two formats:
 - **Hexadecimal Dump:** The numerical representation of each byte in base-16.

- **ASCII Interpretation:** A translation of the hexadecimal bytes into printable ASCII characters. Non-printable characters are typically represented by a dot (.).
- **Purpose:**
 - **Raw Data View:** Allows you to see the exact bytes transmitted over the wire, which is crucial for low-level debugging or identifying obfuscated data.
 - **Correlation:** As you click on fields in the Packet Details Pane, the corresponding bytes in the Packet Bytes Pane are highlighted, illustrating how Wireshark interprets the raw data.
- **Interaction:** You can select segments of the hexadecimal data to see their ASCII translation, or vice-versa.

These three panes work together to provide a comprehensive view of network traffic, from a high-level summary to the individual bits transmitted.

6. Filtering in Wireshark

Filtering is one of Wireshark's most powerful features, essential for narrowing down vast amounts of network traffic to find specific packets of interest. Wireshark distinguishes between two main types of filters:

7. Capture Filters

Definition:

Capture filters are applied *before* the packets are actually saved into the capture file (.pcap or .pcapng). They instruct the network interface card (NIC) driver to discard any packets that do not match the specified filter criteria. Only the packets that pass the filter are written to the capture buffer and subsequently to the disk.

Characteristics:

- **Set before starting the capture:** You define capture filters in the "Capture Options" dialog before you initiate the capture.
- **Based on BPF (Berkeley Packet Filter) syntax:** This is a specialized, relatively low-level syntax. BPF filters are generally less flexible than display filters but are extremely efficient because they operate at the kernel level.
- **Filters packets that are captured and saved to the file:** This is the key difference. Capture filters *reduce the size of the capture file* and *reduce the processing load* on Wireshark during the capture process, especially useful on busy networks.

- **Cannot be changed during capture:** Once a capture is started, the capture filter cannot be modified without stopping and restarting the capture.
- **Limited protocol awareness:** BPF filters generally work at the network (IP) and transport (TCP/UDP) layers. They cannot inspect application-layer content.

Examples of Capture Filters:

Filter	Description
host 192.168.1.5	Capture only packets with either source OR destination IP address 192.168.1.5.
src host 192.168.1.5	Capture only packets <i>from</i> source IP address 192.168.1.5.
dst host 192.168.1.5	Capture only packets <i>to</i> destination IP address 192.168.1.5.
port 80	Capture only HTTP traffic (TCP port 80).
tcp port 443	Capture only HTTPS traffic (TCP port 443). Using tcp is good practice for clarity.
udp port 53	Capture only DNS traffic (UDP port 53).
tcp	Capture only TCP traffic (any TCP port).
udp	Capture only UDP traffic (any UDP port).
icmp	Capture only ICMP (Internet Control Message Protocol) traffic (e.g., ping).
not port 22	Capture all traffic <i>except</i> SSH.
net 192.168.1.0/24	Capture traffic from/to the entire 192.168.1.0/24 subnet.

src net 10.0.0.0/8	Capture traffic originating from the 10.0.0.0/8 network.
ether 00:11:22:33:44:55 host	Capture packets to/from a specific MAC address.

How to Apply:

1. Launch Wireshark.
2. Go to **Capture > Options...** (or click the gear icon on the main screen).
3. In the "Wireshark Capture Interfaces" dialog, select the desired network interface.
4. In the "Capture Filter" field (usually below the interface name), type your BPF filter string.
5. Click "Start" to begin the capture with the filter applied.

Practical Tip: Use capture filters judiciously. While they save space and CPU, if you realize later you needed data that was filtered out, you must perform a new capture. For initial broad captures, it's often better to capture everything and then use display filters for analysis.

8. Display Filters

Definition:

Display filters are applied *after* the packets have already been captured (or opened from a saved file). They do not affect what is stored in the capture file; they only control **which packets are displayed** in the Wireshark GUI's Packet List Pane.

Characteristics:

- **Used after the packets have been captured:** Applied interactively during analysis of a live capture or a loaded .pcap file.
- **Much more powerful and flexible:** Display filters use a richer, more intuitive syntax that allows for very granular filtering based on almost any field Wireshark can dissect (including application-layer data).
- **Filters which packets are displayed in the GUI:** The underlying capture file remains unchanged, containing all original packets. You can change display filters repeatedly to view different subsets of the data.
- **Tshark compatibility:** Display filter syntax is often compatible with tshark (Wireshark's command-line counterpart) for scripting and automated analysis.

Examples of Display Filters:

Filter	Description
ip.src == 192.168.1.1	Show packets where the source IP address is exactly 192.168.1.1. (Note: == for equality, != for not equal).
ip.addr == 192.168.1.1	Show packets where <i>either</i> the source or destination IP address is 192.168.1.1.
tcp.port == 443	Show HTTPS traffic (TCP port 443). Can be tcp.dstport == 443 or tcp.srcport == 443.
http	Show only HTTP packets (Wireshark's dissector for HTTP will automatically match).
dns	Show only DNS queries and responses.
arp	Show only ARP (Address Resolution Protocol) packets.
tcp.flags.syn == 1	Show TCP SYN packets (the first step of a TCP three-way handshake). == 1 means the SYN flag is set.
tcp.flags.ack == 1	Show TCP ACK packets.
tcp.flags.reset == 1	Show TCP RST (Reset) packets (often indicates a connection error or explicit termination).
http.request.method == "GET"	Show only HTTP GET requests.
http.host == "www.example.com"	Show HTTP traffic for a specific hostname.
smb	Show Server Message Block (SMB/CIFS) traffic.

ssl or tls	Show packets related to SSL/TLS handshakes or encrypted data.
contains "password"	Search for packets that contain the ASCII string "password" anywhere in their payload. Case-sensitive by default. matches for regex.
!(http or dns)	Show all traffic <i>except</i> HTTP and DNS.
ip.len > 1500	Show IP packets larger than 1500 bytes (often indicates fragmentation or jumbo frames).

How to Apply:

1. **Type filter expression in the top filter bar:** In the main Wireshark window, locate the "Apply a display filter" input field (usually above the Packet List Pane).
2. **Press Enter:** After typing your filter expression, press Enter. Wireshark will immediately update the Packet List Pane to show only the packets that match your filter.
3. **Color-coding:** The filter bar will change color to indicate validity: green (valid), yellow (warning/partial match), or red (invalid syntax).

Practical Tips: Common Wireshark Use-Cases

Wireshark's flexibility makes it suitable for a wide range of tasks. Here are some common use-cases with relevant display filters:

Task	Display Filter	Explanation
View HTTP GET/POST requests	http.request	Filters for all HTTP requests (GET, POST, PUT, DELETE, etc.). You can then look at the "Info" column for specific methods or filter further using <code>http.request.method == "GET"</code> .

Analyze TCP 3-way Handshake	<code>tcp.flags.syn == 1</code>	Shows all SYN packets, which are the first part of the TCP handshake. You can then follow the TCP stream (Right-click packet > Follow > TCP Stream) to see the entire handshake and subsequent data transfer.
Find login credentials (in plaintext)	<code>http contains "username" or http contains "password" or ftp contains "user" or ftp contains "pass"</code>	Searches the HTTP or FTP payload for common strings indicative of plaintext credentials. Crucial: This only works if credentials are sent unencrypted. For HTTPS, you would need to decrypt SSL/TLS traffic (if you have the server's private key).
Monitor DNS traffic	<code>dns</code>	Shows all DNS queries and responses, useful for troubleshooting name resolution issues or detecting suspicious DNS activity (e.g., DNS exfiltration).
Analyze FTP activity	<code>ftp</code>	Filters for File Transfer Protocol control channel commands. You can follow TCP streams to see file listings and attempted transfers. Remember, FTP is often unencrypted.

Identify common network issues	tcp.analysis.retransmission	Shows TCP retransmissions, often indicating network congestion or packet loss.
	tcp.flags.reset == 1	Shows TCP RST (reset) flags, which can indicate connection problems or abrupt termination.
	icmp	Useful for ping tests and understanding network reachability issues.
View specific host communication	ip.addr == <IP_ADDRESS>	Shows all traffic related to a specific IP address.

Summary of Commands and Tools:

Task	Tool/Command
Enable promiscuous mode (Linux)	ip link set eth0 promisc on
Start Wireshark	wireshark
Save capture file	.pcap or .pcapng
View capture file	Open in Wireshark